CIE-3-Coding Based Activity

A PROJECT REPORT

Submitted by

KRISHNA H CHANDAWAR

Register No: 1RUA25BCA0052
Section: Section A

as part of the Continuous Internal Evaluation
for the course

CS2807 Numerical Methods

Offered in
Semester III
of
BCA (Hons)

School of Computer Science and Engineering
RV University
RV Vidyaniketan, 8th Mile, Mysore Road, Bengaluru, Karnataka, India - 560059

NOVEMBER 2026

1. Problem - 1: Gauss–Jacobi Iterative Method

Problem Statement:

Solve the following system of linear equations using the Gauss–Jacobi iterative method up to four decimal places:

$9x - y + 2z = 18$

$2x + 8y - z = -5$

$x - 2y + 10z = 21$

Use the initial guess $x^{(0)} = 0$, $y^{(0)} = 0$, $z^{(0)} = 0$.

Manual Solution:

Step 1: Check for Strictly Diagonal Dominance

• $|9| = 9 > |-1| + |2| = 3$ (True)

• $|8| = 8 > |2| + |-1| = 3$ (True)

• $|10| = 10 > |1| + |-2| = 3$ (True)

Since the coefficient matrix is diagonally dominant, convergence is guaranteed.

Step 2: Gauss–Jacobi Iteration Formulas

$x^{(k+1)} = (18 + y^{(k)} - 2z^{(k)}) / 9$

$y^{(k+1)} = (-5 - 2x^{(k)} + z^{(k)}) / 8$

$z^{(k+1)} = (21 - x^{(k)} + 2y^{(k)}) / 10$

Step 3: Iteration Calculations (with $x^{(0)}=0$, $y^{(0)}=0$, $z^{(0)}=0$):

• **Iteration 1 (k=0):** $x^{(1)} = 18/9 = 2.0000$, $y^{(1)} = -5/8 = -0.6250$, $z^{(1)} = 21/10 = 2.1000$

• **Iteration 2 (k=1):** $x^{(2)} = (18 - 0.6250 - 4.2000)/9 = 1.4639$, $y^{(2)} = (-5 - 4 + 2.1)/8 = -0.8625$, $z^{(2)} = (21 - 2 - 1.25)/10 = 1.7750$

• **Iteration 3 (k=2):** $x^{(3)} = (18 - 0.8625 - 3.5500)/9 = 1.5097$, $y^{(3)} = -0.7691$, $z^{(3)} = 1.7811$

• **Iteration 4 (k=3):** $x^{(4)} = 1.5187$, $y^{(4)} = -0.7798$, $z^{(4)} = 1.7952$

• **Iteration 5 (k=4):** $x^{(5)} = 1.5144$, $y^{(5)} = -0.7803$, $z^{(5)} = 1.7922$

• **Iteration 6 (k=5):** $x^{(6)} = 1.5150$, $y^{(6)} = -0.7796$, $z^{(6)} = 1.7925$

• **Iteration 7 (k=6):** $x^{(7)} = 1.5150$, $y^{(7)} = -0.7797$, $z^{(7)} = 1.7926$

• **Iteration 8 (k=7):** $x^{(8)} = 1.5150$, $y^{(8)} = -0.7797$, $z^{(8)} = 1.7926$

Final Solution: $x = 1.5150$, $y = -0.7797$, $z = 1.7926$

Python Program:

```
# Gauss-Jacobi Iterative Method
def gauss_jacobi(tol=1e-4, max_iter=50):
    x, y, z = 0.0, 0.0, 0.0
    print(f"Iteration  0: x = {x:.4f}, y = {y:.4f}, z = {z:.4f}")
    for k in range(1, max_iter + 1):
        x_new = (18.0 + y - 2.0 * z) / 9.0
        y_new = (-5.0 - 2.0 * x + z) / 8.0
        z_new = (21.0 - x + 2.0 * y) / 10.0
        print(f"Iteration {k:2d}: x = {x_new:.4f}, y = {y_new:.4f}, z = {z_new:.4f}")
        if max(abs(x_new - x), abs(y_new - y), abs(z_new - z)) < tol:
            print(f"\nConvergence achieved at iteration {k}!")
            print(f"Final Solution: x = {x_new:.4f}, y = {y_new:.4f}, z = {z_new:.4f}")
            break
        x, y, z = x_new, y_new, z_new

gauss_jacobi()
```

OUTPUT:

```
Iteration  0: x = 0.0000, y = 0.0000, z = 0.0000
Iteration  1: x = 2.0000, y = -0.6250, z = 2.1000
Iteration  2: x = 1.4639, y = -0.8625, z = 1.7750
Iteration  3: x = 1.5097, y = -0.7691, z = 1.7811
Iteration  4: x = 1.5187, y = -0.7798, z = 1.7952
Iteration  5: x = 1.5144, y = -0.7803, z = 1.7922
Iteration  6: x = 1.5150, y = -0.7796, z = 1.7925
Iteration  7: x = 1.5150, y = -0.7797, z = 1.7926
Iteration  8: x = 1.5150, y = -0.7797, z = 1.7926

Convergence achieved at iteration 8!
Final Solution: x = 1.5150, y = -0.7797, z = 1.7926
```

2. Problem - 2: Gauss–Seidel Method

Problem Statement:

Solve the following system of linear equations using the Gauss–Seidel method up to four decimal places:

$5x + y - z = 10$

$-x + 6y + 2z = -2$

$2x - y + 7z = 9$

Take the initial guesses: $x^{(0)} = 0$, $y^{(0)} = 0$, $z^{(0)} = 0$.

Manual Solution:

Step 1: Check for Strictly Diagonal Dominance

• $|5| = 5 > |1| + |-1| = 2$ (True)

• $|6| = 6 > |-1| + |2| = 3$ (True)

• $|7| = 7 > |2| + |-1| = 3$ (True)

Convergence is guaranteed.

Step 2: Gauss–Seidel Iteration Formulas (Immediate Substitution)

$x^{(k+1)} = (10 - y^{(k)} + z^{(k)}) / 5$

$y^{(k+1)} = (-2 + x^{(k+1)} - 2z^{(k)}) / 6$

$z^{(k+1)} = (9 - 2x^{(k+1)} + y^{(k+1)}) / 7$

Step 3: Iteration Calculations (with $x^{(0)}=0$, $y^{(0)}=0$, $z^{(0)}=0$):

• **Iteration 1 (k=0):**

$x^{(1)} = (10 - 0 + 0)/5 = 2.0000$

$y^{(1)} = (-2 + 2.0000 - 0)/6 = 0.0000$

$z^{(1)} = (9 - 2(2.0000) + 0)/7 = 5/7 \approx 0.7143$

• **Iteration 2 (k=1):**

$x^{(2)} = (10 - 0 + 0.7143)/5 \approx 2.1429$

$y^{(2)} = (-2 + 2.1429 - 2(0.7143))/6 \approx -0.2143$

$z^{(2)} = (9 - 2(2.1429) - 0.2143)/7 \approx 0.6429$

• **Iteration 3 (k=2):** $x^{(3)} = 2.1714$, $y^{(3)} = -0.1857$, $z^{(3)} = 0.6388$

• **Iteration 4 (k=3):** $x^{(4)} = 2.1649$, $y^{(4)} = -0.1854$, $z^{(4)} = 0.6407$

• **Iteration 5 (k=4):** $x^{(5)} = 2.1652$, $y^{(5)} = -0.1860$, $z^{(5)} = 0.6405$

• **Iteration 6 (k=5):** $x^{(6)} = 2.1653$, $y^{(6)} = -0.1860$, $z^{(6)} = 0.6405$

Final Solution: $x = 2.1653$, $y = -0.1860$, $z = 0.6405$

Python Program:

```
# Gauss-Seidel Iterative Method
def gauss_seidel(tol=1e-4, max_iter=50):
    x, y, z = 0.0, 0.0, 0.0
    print(f"Iteration  0: x = {x:.4f}, y = {y:.4f}, z = {z:.4f}")
    for k in range(1, max_iter + 1):
        x_old, y_old, z_old = x, y, z
        x = (10.0 - y + z) / 5.0
        y = (-2.0 + x - 2.0 * z) / 6.0
        z = (9.0 - 2.0 * x + y) / 7.0
        print(f"Iteration {k:2d}: x = {x:.4f}, y = {y:.4f}, z = {z:.4f}")
        if max(abs(x - x_old), abs(y - y_old), abs(z - z_old)) < tol:
            print(f"\nConvergence achieved at iteration {k}!")
            print(f"Final Solution: x = {x:.4f}, y = {y:.4f}, z = {z:.4f}")
            break

gauss_seidel()
```

OUTPUT:

```
Iteration  0: x = 0.0000, y = 0.0000, z = 0.0000
Iteration  1: x = 2.0000, y = 0.0000, z = 0.7143
Iteration  2: x = 2.1429, y = -0.2143, z = 0.6429
Iteration  3: x = 2.1714, y = -0.1857, z = 0.6388
Iteration  4: x = 2.1649, y = -0.1854, z = 0.6407
Iteration  5: x = 2.1652, y = -0.1860, z = 0.6405
Iteration  6: x = 2.1653, y = -0.1860, z = 0.6405

Convergence achieved at iteration 6!
Final Solution: x = 2.1653, y = -0.1860, z = 0.6405
```

3. Problem - 3: Euler's Method

Problem Statement:
Using Euler's method with step size h = 0.1, estimate y(0.3) for the differential equation:
$dy/dx = x + y^2$,
given y(0) = 1. Compute y(0.1), y(0.2), and y(0.3).

Manual Solution:
Euler's formula: $\mathbf{y_{n+1} = y_n + h \cdot f(x_n, y_n)}$, where $f(x, y) = x + y^2$.

• **Step 1 (n = 0):** $x_0 = 0$, $y_0 = 1$
$f(x_0, y_0) = 0 + (1)^2 = 1.0$
$y_1 = y(0.1) = 1 + 0.1(1.0) =$ **1.1000**

• **Step 2 (n = 1):** $x_1 = 0.1$, $y_1 = 1.1$
$f(x_1, y_1) = 0.1 + (1.1)^2 = 0.1 + 1.21 = 1.3100$
$y_2 = y(0.2) = 1.1 + 0.1(1.3100) =$ **1.2310**

• **Step 3 (n = 2):** $x_2 = 0.2$, $y_2 = 1.231$
$f(x_2, y_2) = 0.2 + (1.231)^2 = 0.2 + 1.515361 = 1.715361$
$y_3 = y(0.3) = 1.231 + 0.1(1.715361) = 1.231 + 0.171536 =$ **1.4025**

Final Values: y(0.1) = 1.1000, y(0.2) = 1.2310, y(0.3) ≈ 1.4025

Python Program:

```
# Euler's Method for ODE
def f(x, y):
    return x + y**2

def euler(x0=0.0, y0=1.0, h=0.1, x_end=0.3):
    steps = int(round((x_end - x0) / h))
    x, y = x0, y0
    print(f"{'Step':<6}{'x_n':<10}{'y_n':<12}{'f(x, y)':<15}{'y_(n+1)':<12}")
    print("-" * 55)
    for i in range(steps):
        slope = f(x, y)
        y_next = y + h * slope
        print(f"{i+1:<6}{x:<10.2f}{y:<12.4f}{slope:<15.6f}{y_next:<12.4f}")
        x = round(x + h, 4)
        y = y_next
    print(f"\nFinal estimated y({x_end}) = {y:.4f}")

euler()
```

OUTPUT:

```
Step  x_n       y_n         f(x, y)        y_(n+1)
-------------------------------------------------------
1     0.00      1.0000      1.000000       1.1000
2     0.10      1.1000      1.310000       1.2310
3     0.20      1.2310      1.715361       1.4025

Final estimated y(0.3) = 1.4025
```

4. Problem - 4: Application of Euler's Method: Newton's Law of Cooling

Problem Statement:

The temperature T of a hot object placed in a room satisfies Newton's Law of Cooling:

$dT/dt = -k(T - T_a)$,

where k = 0.05 per minute and the ambient temperature is T_a = 20°C. If the initial temperature of the object is T(0) = 90°C, use Euler's method with step size h = 4 minutes to estimate the temperature after 8 minutes.

Manual Solution:

Given: dT/dt = -0.05(T - 20), T(0) = 90°C, h = 4 min.

Euler update: $\mathbf{T_{n+1} = T_n + h \cdot f(t_n, T_n)}$

• **Step 1 (t = 0 to 4 min):**

$t_0 = 0$, $T_0 = 90°C$

f(0, 90) = -0.05(90 - 20) = -0.05(70) = -3.5°C/min

$T_1 = T(4) = 90 + 4(-3.5) = 90 - 14.0 =$ **76.0°C**

• **Step 2 (t = 4 to 8 min):**

$t_1 = 4$ min, $T_1 = 76.0°C$

f(4, 76) = -0.05(76 - 20) = -0.05(56) = -2.8°C/min

$T_2 = T(8) = 76.0 + 4(-2.8) = 76.0 - 11.2 =$ **64.8°C**

Final Answer: Temperature after 8 minutes = **64.8°C**

Python Program:

```
# Newton's Law of Cooling using Euler's Method
def newtons_cooling_euler(k=0.05, Ta=20.0, T0=90.0, h=4.0, t_end=8.0):
    t, T = 0.0, T0
    steps = int(round(t_end / h))
    print(f"{'Step':<6}{'Time t (min)':<15}{'Current T (C)':<16}{'dT/dt (C/min)':<16}{'Next T (C)':<12}")
    print("-" * 65)
    for i in range(steps):
        rate = -k * (T - Ta)
        T_next = T + h * rate
        print(f"{i+1:<6}{t:<15.1f}{T:<16.2f}{rate:<16.2f}{T_next:<12.2f}")
        t += h
        T = T_next
    print(f"\nEstimated temperature after {t_end:.0f} minutes: T({t_end:.0f}) = {T:.2f} C")

newtons_cooling_euler()
```

OUTPUT:

```
Step  Time t (min)   Current T (C)   dT/dt (C/min)   Next T (C)
-----------------------------------------------------------------
1     0.0            90.00           -3.50           76.00
2     4.0            76.00           -2.80           64.80

Estimated temperature after 8 minutes: T(8) = 64.80 C
```

5. Problem - 5: Runge–Kutta Second-Order (RK2) Method

Problem Statement:

Using the second-order Runge–Kutta (RK2) method, solve the differential equation:

$dy/dx = \sin(x - y)$,

given $y(0) = 1$. Compute $y(0.3)$ using step size $h = 0.1$.

Manual Solution:

RK2 (Heun's) Formulas:

$k_1 = h \cdot f(x_n, y_n)$

$k_2 = h \cdot f(x_n + h, y_n + k_1)$

$y_{n+1} = y_n + \frac{1}{2}(k_1 + k_2)$

- **Step 1 (x = 0 to 0.1):** $x_0 = 0$, $y_0 = 1$

  $k_1 = 0.1 \cdot \sin(-1 \text{ rad}) = 0.1(-0.841471) = -0.084147$

  $y_0 + k_1 = 0.915853$; $k_2 = 0.1 \cdot \sin(0.1 - 0.915853) = -0.072831$

  $\Delta y_0 = \frac{1}{2}(-0.084147 - 0.072831) = -0.078489$

  $y_1 = y(0.1) = 1 - 0.078489 =$ **0.921511**

- **Step 2 (x = 0.1 to 0.2):** $x_1 = 0.1$, $y_1 = 0.921511$

  $k_1 = 0.1 \cdot \sin(0.1 - 0.921511) = -0.073218$

  $y_1 + k_1 = 0.848293$; $k_2 = 0.1 \cdot \sin(0.2 - 0.848293) = -0.060383$

  $\Delta y_1 = \frac{1}{2}(-0.073218 - 0.060383) = -0.066801$

  $y_2 = y(0.2) = 0.921511 - 0.066801 =$ **0.854710**

- **Step 3 (x = 0.2 to 0.3):** $x_2 = 0.2$, $y_2 = 0.854710$

  $k_1 = 0.1 \cdot \sin(0.2 - 0.854710) = -0.060893$

  $y_2 + k_1 = 0.793817$; $k_2 = 0.1 \cdot \sin(0.3 - 0.793817) = -0.047399$

  $\Delta y_2 = \frac{1}{2}(-0.060893 - 0.047399) = -0.054146$

  $y_3 = y(0.3) = 0.854710 - 0.054146 =$ **0.800564** $\approx$ **0.8006**

Final Result: $y(0.3) \approx$ **0.8006**

Python Program:

```
# Runge-Kutta 2nd Order (RK2) Method
import math

def f(x, y):
    return math.sin(x - y)

def rk2(x0=0.0, y0=1.0, h=0.1, x_end=0.3):
    x, y = x0, y0
    steps = int(round((x_end - x0) / h))
    print(f"{'Step':<6}{'x_n':<8}{'k1':<12}{'k2':<12}{'y_(n+1)':<12}")
    print("-" * 55)
    for i in range(steps):
        k1 = h * f(x, y)
        k2 = h * f(x + h, y + k1)
        y_next = y + 0.5 * (k1 + k2)
        x = round(x + h, 4)
        print(f"{i+1:<6}{x:<8.2f}{k1:<12.6f}{k2:<12.6f}{y_next:<12.6f}")
        y = y_next
    print(f"\nFinal estimated y({x_end}) = {y:.4f}")

rk2()
```

OUTPUT:

```
Step  x_n     k1          k2          y_(n+1)
-------------------------------------------------------
1     0.10    -0.084147   -0.072831   0.921511
2     0.20    -0.073218   -0.060383   0.854711
3     0.30    -0.060893   -0.047399   0.800565

Final estimated y(0.3) = 0.8006
```

6. Problem - 6: Application of RK2 Method: Salt Mixture Problem

Problem Statement:
A tank initially contains 80 liters of pure water. Brine containing 3 grams of salt per liter flows into the tank at a rate of 4 liters per minute, and the well-mixed solution flows out at the same rate. Let y(t) denote the amount of salt, in grams, in the tank at time t minutes. The rate of change of salt is given by:

dy/dt = 12 - y/20.

Using the second-order Runge–Kutta method with step size h = 5 minutes, estimate the amount of salt in the tank after 15 minutes, given y(0) = 0.

Manual Solution:
Model: dy/dt = 12 - 0.05y, y(0) = 0, h = 5 min.
RK2 formula: $y_{n+1} = y_n + ½(k_1 + k_2)$

• **Step 1 (t = 0 to 5 min):** $t_0 = 0$, $y_0 = 0$
$k_1 = 5(12 - 0) = 60.0000$
$k_2 = 5(12 - 0.05(60)) = 5(9) = 45.0000$
$y_1 = y(5) = 0 + ½(60 + 45) =$ **52.5000 g**

• **Step 2 (t = 5 to 10 min):** $t_1 = 5$, $y_1 = 52.5$
$k_1 = 5(12 - 0.05(52.5)) = 5(9.375) = 46.8750$
$y_1 + k_1 = 99.375$; $k_2 = 5(12 - 0.05(99.375)) = 35.1562$
$y_2 = y(10) = 52.5 + ½(46.8750 + 35.1562) =$ **93.5156 g**

• **Step 3 (t = 10 to 15 min):** $t_2 = 10$, $y_2 = 93.515625$
$k_1 = 5(12 - 0.05(93.515625)) = 36.6211$
$y_2 + k_1 = 130.1367$; $k_2 = 5(12 - 0.05(130.1367)) = 27.4658$
$y_3 = y(15) = 93.515625 + ½(36.6211 + 27.4658) =$ **125.5591 g**

Final Answer: Amount of salt after 15 minutes = **125.5591 grams**
(Exact analytical solution: $y(15) = 240(1 - e^{-0.75}) \approx 126.6320$ g, relative error ≈ 0.85%).

Python Program:

```
# Application of RK2 - Salt Mixture Problem
import math

def dydt(t, y):
    return 12.0 - y / 20.0

def salt_mixture(h=5.0, t_end=15.0):
    t, y = 0.0, 0.0
    steps = int(round(t_end / h))
    print(f"{'Step':<6}{'Time t (min)':<14}{'k1 (g)':<14}{'k2 (g)':<14}{'Salt y(t) (g)':<14}")
    print("-" * 65)
    for i in range(steps):
        k1 = h * dydt(t, y)
        k2 = h * dydt(t + h, y + k1)
        y_next = y + 0.5 * (k1 + k2)
        t += h
        print(f"{i+1:<6}{t:<14.1f}{k1:<14.4f}{k2:<14.4f}{y_next:<14.4f}")
        y = y_next
    print(f"\nEstimated salt after {t_end:.0f} minutes: y({t_end:.0f}) = {y:.4f} grams")
    exact = 240.0 * (1.0 - math.exp(-t_end / 20.0))
    print(f"Exact analytical value: y_exact = {exact:.4f} grams")

salt_mixture()
```

OUTPUT:

```
Step  Time t (min)  k1 (g)        k2 (g)        Salt y(t) (g)
-----------------------------------------------------------------
1     5.0           60.0000       45.0000       52.5000
2     10.0          46.8750       35.1562       93.5156
3     15.0          36.6211       27.4658       125.5591

Estimated salt after 15 minutes: y(15) = 125.5591 grams
Exact analytical value: y_exact = 126.6320 grams
```